One of the first challenges I had with Codex was understanding what it was asking me to approve. In my first few prompts, I found it really hard to know what I was actually agreeing to when Codex asked if it was okay to do something. I ended up using ChatGPT to help decipher what it was asking me to approve. Later, it stopped asking me for approval for certain actions, which was also a little confusing because I did not really understand why it needed approval before but no longer did.

I also found that Codex initially built things in a way more complicated way than I would have. It would sometimes build out functions or more generalized code when I would have just used a built-in R function like `summary()`. I think some of this was probably intended to make the code more reusable or reproducible, but for a fairly simple analysis it made it way harder for me to understand what was happening. It also does not always name things the same way I would. When I write my own code, I know what I named something and where I used it, but when I did not write the code, it was harder to follow what was getting reused where. Similarly, when I wanted to remove something that I did not think was necessary or was not really adding anything to the analysis, it was difficult to know if removing that code would cause a problem somewhere else later. That is something I am concerned about going into our larger project, where the oil wells analysis and all of the information we are working with is already complicated. Adding code that is more complicated than it needs to be could easily make it even harder to work through and understand what is actually happening in the analysis.

As I worked through the analysis, I started being more specific in my prompts about wanting simple, readable R code and not adding unnecessary functions or complexity. That did help with the later outputs, but it also showed me that I need to put a lot of bounds around what I am asking Codex to do so that I will actually be able to read and evaluate the output. It almost felt like I needed to already know how I would approach something so that I could be specific enough about how I wanted Codex to do it. Otherwise, I might get something that worked but was much harder for me to follow than it needed to be, which also doesn't really feel like it's saving me time if I already know how to do it.

For a small analysis like this, I do not think Codex really saved me any time. Sure, it reduced the time spent actually typing code, but shifted a lot of that effort into crafting prompts and reviewing code that was harder to review because I did not write it, did not choose how things were named, and did not always know why it did things a certain way. I do think I got a little better at working with it as I went. Since I did this over a few different Codex sessions, I also saw the value of having a good reusable context prompt rather than trying to recreate everything each time. Going forward, I would probably have a standard prompt to kick off each session with the ground rules about what I am trying to achieve, how I want

the code written, and how I want Codex to work, and then add the specific task after that. I think that would make it easier to use without having to recreate those expectations multiple times.